

Trabajo de Cursada

Computación Evolutiva (2025)



Problema del Viajante (TSP)



Integrantes:

- González Tesoro, Joaquín. jgonzaleztesoro@alumnos.exa.uncen.edu.ar.
- Roumec, Iñaki. iroumec@alumnos.exa.uncen.edu.ar.

Índice

Introducción.....	3
Elección del Algoritmo Evolutivo.....	3
Definición de Componentes.....	3
Esquema de Ejecución General del Algoritmo.....	3
Representación o Codificación de las Soluciones.....	4
Proceso de Decodificación.....	4
Función de Evaluación o de Fitness.....	4
Proceso de Generación de la Población Inicial.....	4
Proceso de Selección de Padres.....	5
Operadores de Cruce Genético.....	5
Operadores de Mutación Genética.....	6
Enfoque de Búsqueda Local.....	6
Proceso de Selección de Sobrevivientes, o Proceso de Reinserción.....	6
Análisis de Valores.....	7
Probabilidad de Cruce.....	7
Probabilidad de Mutación.....	7
Tamaño de la Población Inicial.....	7
Condición de Finalización para la Ejecución del Algoritmo.....	7
Referencias.....	8

Introducción

En este informe, se realiza la elección de un algoritmo evolutivo y la definición de sus componentes, en base a la literatura y a lo explicado en la cursada, para resolver el **Problema del Viajante** o **Travelling Salesman Problem (TSP)**. Este es un problema de optimización **NP-Hard** y representa a diversos problemas de optimización que ocurren en una amplia variedad de dominios reales de aplicación. A continuación, se enuncia el problema:

Dado un conjunto de N ciudades, y el costo del viaje entre cada par de esas ciudades, encontrar el camino de mínimo costo que permite visitar todas las ciudades del conjunto (pasando solamente una vez por cada ciudad) y retornar a la ciudad de partida.

La consigna no especifica si el problema a resolver es asimétrico (el costo de ir de la ciudad i a la ciudad j puede no ser el mismo que el de ir de la ciudad j a la ciudad i), abreviado como **ATSP**, o simétrico (el costo de ir de la ciudad i a la ciudad j es el mismo que el de ir de la ciudad j a la ciudad i), también llamado **Problema del Viajante de Steiner** y abreviado **STSP**. Se realizará el planteamiento considerando la primera variante: el ATSP.

Elección del Algoritmo Evolutivo

Los **algoritmos meméticos o memetic algorithms (MA/GLS)** combinan **algoritmos evolutivos o evolutive algorithms (EA)** con búsqueda local intensiva y han demostrado ser muy efectivos en problemas de optimización combinatoria como el problema del viajante [1]. Merz y Freisleben (2001) reportan que sus MA con recombinación genética obtienen soluciones casi óptimas en instancias de TSP (y ATSP) de decenas de miles de ciudades [1]. En particular, observaron que incorporar búsqueda local mejora notablemente los resultados de un GA puro. Por comparación, estudios aplicados al TSP clásico indican que los **algoritmos de optimización de colonias de hormigas o ant colony optimization algorithms (ACO)** producen, en promedio, mejores recorridos que un GA sin mejoras locales, aunque suele requerir más tiempo de cómputo [5]. A su vez, estudios más recientes demuestran la efectividad de metaheurísticas bio-inspiradas como *particle swarm optimization*, *firefly algorithm* y *bat algorithm* [6]. No obstante, puesto a que estos no fueron abordados en el curso, no se tomaron en consideración.

Dado que los MA suelen superar o igualar a un GA puro, aprovechando ventajas exploratorias de GA y explotativas de búsqueda local, posicionándose como un método balanceado entre calidad y eficiencia para TSP/ATSP [1], **es la opción por la que se optó para abordar el problema.**

Definición de Componentes

A continuación, se describe la definición de las componentes del algoritmo elegido.

Esquema de Ejecución General del Algoritmo

El esquema de ejecución general del algoritmo define su ciclo de vida. Existen dos esquemas de ejecución principales:

- **Modelo Generacional:** Consiste en la “renovación total”. Se crea una población completa de hijos (P’) y esta reemplaza a la población de padres (P). El ciclo es por “bloques”.
- **Modelo *Steady-State* (Estado Estacionario):** Consiste en la “renovación continúa” o “renovación incremental”. En cada iteración se crean uno o dos hijos e inmediatamente se intenta insertarlos en la población actual.

Para MA, el *Steady-State*, junto a esquemas ($\mu+\lambda$) superpuestos (los cuales no se consideraron ya que no se abordaron en el curso), suele ser preferido porque preserva mejor las estructuras de alta calidad encontradas por la búsqueda local, además de que evita grandes oscilaciones y aprovecha rápidamente las mejoras [1]. Por consiguiente, **fue la opción que se eligió.**

Representación o Codificación de las Soluciones

Se propone como representación o codificación de las soluciones el uso de una **permutación**. En esta, la **adyacencia conforma una restricción** fundamental, puesto a que dictamina el orden en el que las ciudades son recorridas. Es decir, **no importa el lugar en el que una ciudad aparezca, sino junto a qué ciudades aparece**. Esta es la representación estándar del problema y se la denomina *path representation*. Es natural porque refleja directamente la estructura de la solución (un ciclo hamiltoniano) y evita duplicados. Por ello, prácticamente todos los trabajos la usan [1].

En *path representation*, la primera ciudad se conecta implícitamente con la última. Esto implica que: [3, 2, 0, 1] es equivalente a [0, 1, 3, 2], puesto a que proponen el mismo recorrido. Si bien son equivalentes, ambas permanecen en la población, ya que, debido a que no representan la misma permutación, la aplicación de operadores pueden llevarlas a lugares distintos del espacio de búsqueda, aumentando la exploración del algoritmo.

Proceso de Decodificación

Dado que se utiliza una codificación por permutación directa, el genotipo corresponde unívocamente al fenotipo. El proceso de decodificación consiste simplemente en interpretar la secuencia de enteros como el orden de visita de las ciudades, cerrando el ciclo desde la última ciudad a la primera para formar el recorrido completo. **Una decodificación adicional podría considerarse si se tiene en cuenta el mapeo de los números enteros al nombre de las ciudades correspondientes.**

Función de Evaluación o de *Fitness*

Siguiendo la perspectiva evolutiva de la “supervivencia del más apto”, se considera al problema como un problema de maximización. Con esto en cuenta, se define al *fitness* como: $1/L$, donde L es la distancia recorrida al realizar el recorrido. De esta forma, **camino más cortos, resultan en un mayor *fitness* y un mayor *fitness* resulta en una mejor solución al problema.**

Proceso de Generación de la Población Inicial

En la literatura académica, la inicialización puramente aleatoria es el estándar para probar la robustez del algoritmo (para demostrar que no depende de la suerte inicial) [1]. Sin embargo, se halló que, en algoritmos meméticos para TSP, a menudo se usa la **inicialización híbrida o mixta**, complementándose la inicialización aleatoria con heurísticas, puesto a que presenta buenos resultados.

Una heurística muy utilizada es la del **vecino más cercano** o **nearest neighbor (NN)**, la cual consiste en construir recorridos agregando siempre la ciudad más cercana no visitada [7]. Alkafawen et al. (2021) señalan que el NN es “la técnica de población inicial más común” en GA para TSP [7]. De hecho, Merz y Freisleben encontraron que los recorridos iniciales obtenidos por un heurístico *greedy* “son los más adecuados para la búsqueda local”, porque contienen errores explotables [1].

Adoptando esta visión, se decidió generar la población inicial de la siguiente manera:

- Un **20 %** usando la **heurística NN**, con el objetivo de comenzar con una población con individuos fuertes.
- Y el **80 %** restante de forma totalmente **aleatoria**, con el objetivo de asegurar diversidad genética.

De esta forma, los aleatorios aportan exploración, mientras que los heurísticos aportan buenos puntos de partida para refinamiento local [1].

Proceso de Selección de Padres

En GA para TSP lo habitual es usar **selección por ruleta** (proporcional a la aptitud) o **por torneo** (seleccionar los mejores de grupos aleatorios). Ambos métodos son estándar: por ejemplo, Miller y Goldberg (1995) indican que la selección por torneo es “útil y robusta” y común en GA [2]. Según afirman, el torneo permite controlar fácilmente la presión selectiva (tamaño del torneo) y suele mantener más diversidad. La ruleta es la selección clásica por probabilidad según *fitness*. En la práctica se puede usar cualquiera; muchos estudios usan torneo por su simplicidad y buen comportamiento con funciones ruidosas [2], mientras que la ruleta se usa en contextos más tradicionales. En todo caso, no hay rechazo generalizado de ninguno, sino preferencia según experimentos. Con esto en cuenta, y considerando que en la teoría de la cursada se mencionó que la selección por ruleta es, en general, más ineficiente, **se decidió optar por torneo para abordar el problema.**

Operadores de Cruce Genético

Para representaciones permutacionales se emplean cruces especiales que preservan elementos. Se evaluaron distintas alternativas:

- El **PMX (Partially-Matched Crossover)** fue introducido por Goldberg y Lingle (1985) y es muy popular en TSP [1]; copia un segmento de un padre y hace mapeos para construir la descendencia.
- El **OX1 (Order Crossover)** de Davis (1985) también es habitual, preserva el orden relativo de las ciudades al rellenar espacios vacíos.
- El **crossover de recombinación de aristas (Edge Recombination Operator, ERO)**, propuesto por Whitley (1989), crea un hijo construyendo las listas de vecinos de los padres y eligiendo aristas compartidas [8]. Es adecuado para TSP porque mantiene información de adyacencia. Merz y Freisleben (2001) notan que sin mejora local, ERO suele superar a PMX (menor tasa de aristas “foráneas”) [1], pero con búsqueda local la diferencia se invierte (PMX puede lograr mejores soluciones con menos perturbaciones) [1].
- El **DPX (Distance-Preserving Crossover)**, propuesto por Merz y Freisleben, está diseñado para meméticos: fuerza incluir aristas foráneas y generar un hijo equidistante a ambos padres

[1]. DPX solo resulta rentable si se dispone de mejora local luego (de lo contrario inclusiones arbitrarias dañan la calidad) [1].

- Finalmente, se halló que, en un *paper* del año anterior, se propuso un operador con resultados mejores a los anteriores bajo ciertas condiciones: el ***Dynastic Potential Crossover***, el cual también es abreviado **DPX**. Este se basa en el concepto de ***dynastic potential***, el cual se define como el conjunto de todas las soluciones que pueden ser generadas por un operador de cruce a partir de dos padres [9].

Si bien el *distance-preserving crossover* o el *dynastic potential crossover* parecieran ser los mejores, dada su complejidad de implementación y a que no fueron abordados en el curso, fueron descartados. En su lugar, **se optó por el operador PMX**.

Operadores de Mutación Genética

Para permutaciones los operadores de mutación más sencillos y conocidos son el **swap** (intercambio de dos genes), la **inversión** (o 2-opt, invertir el orden de un subsegmento) y el **scramble** (escoger un subconjunto y permutar sus elementos aleatoriamente) [3]. Dado que el *swap* es el más común en codificaciones permutacionales [3], **se optó por este para enfrentar el problema**.

Enfoque de Búsqueda Local

Una vez generado un hijo, tras cruce y mutación, debido al uso de un algoritmo memético, es necesario aplicar un algoritmo rápido para llevar a ese individuo a su óptimo local antes de devolverlo a la población, lo que se conoce como **búsqueda local** (por ejemplo, 2-opt o inversión). Para ello hay dos enfoques: el **Lamarckiano**, donde después de optimizar localmente se reemplaza el genotipo por la versión mejorada, y el **Baldwiniano**, donde solo se actualiza la aptitud sin cambiar el genotipo original.

Según la literatura, el enfoque Lamarckiano es el más común y efectivo en MA [10]. Tal como explican Xin et al. (2016), en el aprendizaje Lamarckiano “el genotipo refleja el resultado de la mejora local” (se reingresa el individuo mejorado) mientras que en el Baldwiniano solo se altera la aptitud [10]. La mayoría de las aplicaciones exitosas de MA usan el paradigma Lamarckiano, porque preserva mejoras concretas para las generaciones futuras [10]. Considerando lo anterior, **se decidió por aplicar un enfoque Lamarckiano**. Debido a restricciones en la longitud del informe y sus incisos, se posterga el desarrollo de la búsqueda local al informe final, junto a la implementación del algoritmo.

Proceso de Selección de Sobrevivientes, o Proceso de Reinserción

Dado que el esquema de ejecución elegido es *Steady-State*, el proceso de selección de sobrevivientes debe evaluar: ¿a quién mata el hijo nuevo? Existen dos procesos posibles:

- ***Replace Worst***: Se reemplaza al peor de la población, eliminando así a individuos de menor calidad. Es el enfoque que Metz y Freisleben sugieren optar y utilizan en sus ejemplos [1].
- ***Tournament Replacement* o reemplazo mediante torneo de reinserción**: El hijo compite con un subgrupo y reemplaza al perdedor. En otras palabras, se selecciona a un pequeño grupo al azar y se sustituye al peor entre ellos por el nuevo hijo.

De acuerdo a lo investigado, no existe un consenso único acerca del proceso más recomendable. Ambos enfoques se utilizan en la práctica. Lo importante es mantener la diversidad y no eliminar todos los óptimos prematuramente. Considerando todo lo anterior, **se decidió optar por el enfoque de tournament, puesto a que preserva mejor la diversidad.**

Análisis de Valores

A continuación, se realiza un análisis acerca de cuáles son los valores más convenientes en relación a los parámetros. Debido a que en el siguiente trabajo se realizarán evaluaciones al respecto, se proporcionan rangos teóricos a probar y no, valores puntuales (puesto a que eso requiere experimentación).

Probabilidad de Cruce

Es conveniente una probabilidad alta (entre 0.8 y 0.9) para asegurar recombinación frecuente de rasgos prometedores. Este rango alto es común en la literatura, puesto a que favorece la explotación de buenas soluciones. Aunque no se halló un estudio que dé valores exactos para ATSP, en las referencias que se citan en el informe se recomiendan rangos elevados (**entre 0.6 y 0.9**) como punto de partida.

Probabilidad de Mutación

Las fuentes halladas indican valores “típicos” **entre 0.001 y 0.1** [4], con 0.01 como estándar para rutas grandes. Una probabilidad de mutación mayor (hasta 0.1) puede ayudar a escapar de óptimos locales en búsqueda memética, pero se debe ser cauteloso en su uso para no destruir la estructura buena. Se sugiere una mutación inversamente proporcional al tamaño de la población o empezar con una probabilidad de mutación relativamente baja (por ejemplo, 0.05) y ajustarla si hay estancamiento [4].

Tamaño de la Población Inicial

Los MA suelen usar poblaciones más pequeñas que GA puros debido al costo de las búsquedas locales (en las que se evalúa cada individuo). Merz y Freisleben mencionan que es común usar **entre 10–40 individuos** en un MA, ya que poblaciones mayores harían inviable el tiempo de cómputo. Adicionalmente, justifican que esto es apropiado, puesto a que se aumenta el número de iteraciones posibles con el presupuesto dado, sin comprometer la diversidad, puesto a que la mejora local compensa en parte el menor tamaño [1]. Sin embargo, han pasado varios años desde la publicación del *paper*, por lo que **este número podría extenderse considerando la capacidad de cómputo actual.**

Condición de Finalización para la Ejecución del Algoritmo

Además de un número máximo de generaciones, se suele emplear un criterio de convergencia: detener cuando no hay mejora en N iteraciones consecutivas. En la literatura (e.g. Bhandari et al.) se le conoce a este método como “**K-iterations**” o **parada por convergencia**: la ejecución del algoritmo se para si en K generaciones seguidas no mejora el mejor fitness [4]. Este criterio refleja que el algoritmo ha convergido a un óptimo local y conviene interrumpirlo. Según los experimentos, **se puede fijar $N = 50$ o 100 generaciones sin mejora.**

Referencias

- [1] P. Merz y B. Freisleben, “Memetic Algorithms for the Traveling Salesman Problem,” *Complex Systems*, vol. 13, pp. 297–345, 2001. [En línea]. Disponible: <https://content.wolfram.com/sites/13/2018/02/13-4-1.pdf>. [Consultado: 5-dic-2025].
- [2] B. L. Miller y D. E. Goldberg, “Genetic Algorithms, Tournament Selection, and the Effects of Noise,” *Complex Systems*, vol. 9, pp. 193–212, 1995. [En línea]. Disponible: <https://wpmedia.wolfram.com/sites/13/2018/02/09-3-2.pdf>. [Consultado: 5-dic-2025].
- [3] “Genetic Algorithms — Mutation,” *Tutorialspoint*. [En línea]. Disponible: https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_mutation.htm. [Consultado: 5-dic-2025].
- [4] “Chapter 3: Mutation and Its Role,” *Algorithm Afternoon* (Genetic Algorithm Afternoon). [En línea]. Disponible: https://algorithmafternoon.com/books/genetic_algorithm/chapter03/. [Consultado: 5-dic-2025].
- [5] H. H. A. Mukhairez y A. Y. A. Maghari, “Performance Comparison of Simulated Annealing, GA and ACO Applied to TSP,” *International Journal of Information and Communication Research (IJICR)*, vol. 6, 2015. Hosam H. A. Mukhairez, Ashraf Y. A. Maghari. [En línea]. Disponible: <https://infonomics-society.org/wp-content/uploads/ijicr/published-papers/volume-6-2015/Performance-Comparison-of-Simulated-Annealing-GA-and-ACO-Applied-to-TSP.pdf>. [Consultado: 5-dic-2025].
- [6] “Traveling Salesman Problem,” *ScienceDirect Topics* (Elsevier). [En línea]. Disponible: <https://www.sciencedirect.com/topics/computer-science/traveling-salesman-problem>. [Consultado: 5-dic-2025].
- [7] E. Alkafaween, A. B. A. Hassanat y S. Tarawneh, “Improving Initial Population for Genetic Algorithm Using the Multi Linear Regression Based Technique (MLRBT),” *Communications (University of Zilina)*, vol. 23, no. 1, pp. E1–E10, 2021. [En línea]. Disponible: <https://pdfs.semanticscholar.org/d49e/0d39594531be64455c003a7bb860ea3e019c.pdf>. [Consultado: 5-dic-2025].
- [8] “Edge Recombination Operator,” *Wikipedia*. [En línea]. Disponible: https://en.wikipedia.org/wiki/Edge_recombination_operator. [Consultado: 5-dic-2025].
- [9] Chicano, G. Ochoa, D. Whitley y R. Tinós, “Dynastic Potential Crossover Operator,” *arXiv preprint arXiv:2402.03918*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.03918>. [Accessed: 5-dic-2025]
- [10] M. N. Le, Y.-S. Ong, Y. Jin y B. Sendhoff, “Lamarckian Memetic Algorithms: Local Optimum and Connectivity Structure Analysis,” preprint aceptado en *Memetic Computing*, 2009. [En línea]. Disponible: <https://www.honda-ri.de/pubs/pdf/1352.pdf>. [Consultado: 5-dic-2025].